

一、实验内容

本实验基于微信小程序开发了一款推箱子益智游戏。采用愤怒的小鸟角色及场景素材，玩家通过方向键或屏幕手势控制主角移动并推动箱子，将箱子推至目标点即可消灭该处的敌方角色并通关。小程序包含首页与游戏页两个页面，共设置八个关卡。关卡数据由本地 JS 文件提供，通关进度、最佳纪录与关卡解锁状态均通过微信本地存储持久化。此外，游戏内置了剧情对话、动画特效、音效与背景音乐。

1. 首页：关卡选择、通关进度与破解版解锁

首页采用沉浸式自定义导航栏，并依据微信右上角胶囊按钮的位置动态预留右侧空白，避免自定义按钮被遮挡。导航栏左侧为游戏标题，右侧为“破解版”与“重置进度”两个功能按钮。导航栏下方为一张游戏简介卡片，用于说明玩法。

关卡列表以两列卡片形式展示。每张关卡卡片包含关卡预览图、关卡编号、关卡名称与最佳纪录信息。已通关关卡显示最佳步数，未通关关卡显示“未通关”，未解锁关卡整体置灰并叠加锁形遮罩。每通关一关即自动解锁下一关，进度写入本地存储，重新进入页面后数据不会丢失。

“破解版”按钮提供一键解锁全部关卡的功能，但需先正确回答一个问题。该问题为“主角的敌人是谁”，正确选项为“猪”。回答正确后写入存储并刷新关卡列表，全部关卡同时解锁；回答错误则提示可重新尝试。“重置进度”按钮用于清空全部通关进度与最佳纪录。

下图为首页整体效果，包含导航栏、简介卡片与两列关卡列表：



下图为破解版解锁问答弹窗，回答正确后一键解锁全部关卡：

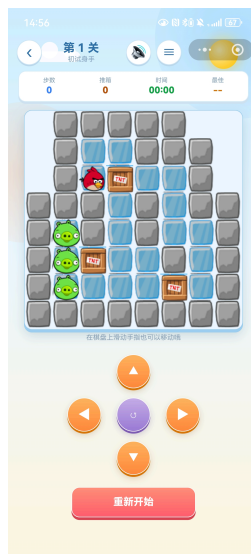


2. 游戏页：Canvas 2D 棋盘渲染

游戏页顶部为自定义导航栏，包含返回按钮、当前关卡信息、声音开关与选关按钮。导航栏下方为统计栏，实时显示步数、推箱次数、用时与最佳步数。

棋盘为 8×8 画布，使用新版 Canvas 2D 接口绘制，替代了教材中已废弃的旧版画布接口。棋盘内的地板、石墙、目标点、箱子与主角均使用项目提供的图片素材绘制。图片通过画布节点预加载后调用绘制接口输出，并按设备像素比进行高清适配。若图片加载失败，则自动回退为代码矢量绘制，避免画面空白。

下图为游戏页整体效果，包含统计栏、棋盘、方向键与重新开始按钮：



互动时棋盘上可见推箱尘土、主体上下左右推动时形态、到达目标点时动态出现语气词气泡等





3. 交互与动画特效

移动操作支持两种方式，即点击方向键与在棋盘上滑动屏幕。方向键区域包含上、下、左、右四个方向键与中间的撤销键，四个方向键采用统一的圆形配色，排布整齐。

游戏基于 requestAnimationFrame 建立连续渲染循环，统一驱动移动动画与各类特效。主角移动时平滑滑动并伴随跳跃拉伸效果；向右移动保持默认朝向，向左移动时执行平滑镜像翻转。主角图片上叠加一对可随移动方向转动的眼睛。由于主角素材为侧身绘制，开发时先对图片进行逐像素分析，定位眼睛的真实坐标，并将眼睛绘制纳入缩放上下文，确保跳跃与翻转过程中眼睛始终贴合角色。

目标点上的敌方角色会原地呼吸脉动。箱子被推至目标点后，箱子角上的星形标记以回弹动画出现并持续呼吸缩放。每次推动箱子都会产生尘土粒子与扩散波纹，推动过程中持续冒烟。箱子压上目标点的瞬间，目标点上方会弹出语气词气泡，气泡内容从多组语气词中随机选取，并带有弹出与淡出动画。撤销键可逐步回退操作，重新开始按钮可将当前关卡恢复至初始状态。

4. 剧情系统

每关开场均会弹出剧情弹窗。弹窗中主角位于气泡旁，气泡内展示该关的剧情台词，各关台词互不相同。点击“开始游戏”按钮后进入棋盘。通关时主角以第一人称说出祝贺台词。剧情台词与祝贺台词统一存放于 utils/data.js 的各关卡对象中。

下图为第 1 关开场剧情弹窗，包含主角、对话气泡与开始游戏按钮：



5. 通关界面

通关后弹出庆祝面板。面板中主角位于左侧并上下浮动，右侧气泡展示第一人称祝贺台词；“恭喜通关！”标题以大号号回弹动画显示；若刷新最佳纪录则显示对应徽章；统计栏中的步数、推箱次数与用时数字会滚动增长；面板背后有全屏彩色纸屑飘落，同时触发震动反馈与通关音效。

下图为通关界面，包含主角祝贺、大号动态标题、数字滚动统计与全屏彩带：



6. 音效与背景音乐

游戏内置程序合成的音效与循环背景音乐，通过微信音频接口播放。音效包括移动、推箱、撞墙、点击、目标角色被压住与通关共六种，背景音乐为欢快循环的旋律。背景音乐在用户点击“开始游戏”按钮后开始播放，以规避平台对自动播放的限制。导航栏的声音开关可一键控制全部声音，选择结果写入本地存储。页面切换至后台时自动暂停音乐，返回时恢复，离开页面时销毁音频上下文。

7. 选关面板

游戏页的选关按钮可打开选关面板，以网格形式展示全部关卡，包含关卡序号、名称与完成状态。点击已解锁关卡可跳转切换，未解锁关卡点击时给出提示。

下图为选关面板：



8. 核心代码

- app.json 全局配置，声明两个页面并启用沉浸式自定义导航栏。

```
{
  "pages": [
    "pages/index/index",
    "pages/game/game"
  ],
  "window": {
    "navigationStyle": "custom",
    "backgroundColor": "#d9f1ff",
    "backgroundTextStyle": "light"
  },
  "style": "v2",
  "sitemapLocation": "sitemap.json"
}
```

- utils/data.js 定义关卡数据，包含地图、剧情台词与通关台词。地图数值含义为：0 表示地图外空地、1 表示墙壁、2 表示地板、3 表示目标点、4 表示箱子、5 表示主角。

```
const levels = [
  {
    name: '初试身手',
    story: '请帮帮 🐷!',
    winLine: '谢谢，你真厉害！猪猪全被炸光啦！',
    map: [
      [0,1,1,1,1,1,0,0],
      [0,1,2,2,1,1,1,0],
      [0,1,5,4,2,2,1,0],
      [1,1,1,2,1,2,1,1],
      [1,3,1,2,1,2,2,1],
      [1,3,4,2,2,1,2,1],
      [1,3,2,2,2,4,2,1],
      [1,1,1,1,1,1,1,1]
    ]
  }
]
```

```
// .....其余 7 关
];
module.exports = { levels, maps: levels.map(item => item.map) };
```

- pages/index/index.js 实现破解版解锁，回答正确后一键解锁全部关卡。

```
answerUnlock: function (e) {
  const correct = e.currentTarget.dataset.correct
  if (correct === true || correct === 'true') {
    const progress = wx.getStorageSync(STORAGE_KEY) || { unlocked: 1, best: {} }
    progress.unlocked = data.levels.length
    wx.setStorageSync(STORAGE_KEY, progress)
    this.setData({ showUnlock: false })
    this.loadProgress()
    wx.showToast({ title: '破解成功! 全部关卡已解锁 🎉', icon: 'none' })
  } else {
    wx.showToast({ title: '答错啦, 再想想~', icon: 'none' })
  }
}
```

- pages/game/game.js 初始化新版 Canvas 2D 画布，并按设备像素比高清适配。

```
initCanvas: function () {
  const self = this
  const query = wx.createSelectorQuery()
  query.select('#gameCanvas').fields({ node: true, size: true }).exec(function (res) {
    if (!res || !res[0] || !res[0].node) return
    const canvas = res[0].node
    const ctx = canvas.getContext('2d')
    const info = wx.getWindowInfo ? wx.getWindowInfo() : wx.getSystemInfoSync()
    const dpr = info.pixelRatio || 2
    canvas.width = res[0].width * dpr
    canvas.height = res[0].height * dpr
    ctx.scale(dpr, dpr)
    self.canvas = canvas
    self.ctx = ctx
    self.cell = Math.floor(Math.min(self.cw, self.ch) / 8)
    self.initMap()
    self.loadIcons()
    self.startRenderLoop()
  })
}
```

- pages/game/game.js 实现移动与推箱的核心逻辑。

```
tryMove: function (dr, dc) {
  if (this.animating || this.won || !this.map || !this.player) return
  const r = this.player.r + dr
  const c = this.player.c + dc
  if (r < 0 || r >= 8 || c < 0 || c >= 8) return
  if (this.map[r][c] === 1) {
```

```

    this.playSfx('bump'); this.vibrate('light'); return
  }
  let boxMove = null
  if (this.box[r][c] === 4) {
    const br = r + dr, bc = c + dc
    if (br < 0 || br >= 8 || bc < 0 || bc >= 8) return
    if (this.map[br][bc] === 1 || this.box[br][bc] === 4) return
    boxMove = { fr: r, fc: c, tr: br, tc: bc }
  }
  this.history.push({ r: this.player.r, c: this.player.c,
    boxes: this.cloneBoxes(), moves: this.moves, pushes: this.pushes })
  if (boxMove) {
    this.box[boxMove.fr][boxMove.fc] = 0
    this.box[boxMove.tr][boxMove.tc] = 4
    this.pushes += 1
    this.spawnPushFx(boxMove, dr, dc)
  }
  this.player.r = r; this.player.c = c
  this.dir = dr === -1 ? 0 : dr === 1 ? 1 : dc === -1 ? 2 : 3
  this.moves += 1
  if (!this.timerStarted) this.startTimer()
  this.setData({ moves: this.moves, pushes: this.pushes })
  this.playSfx(boxMove ? 'push' : 'move')
  this.anim = { t0: Date.now(), dur: 160,
    from: { r: this.player.r - dr, c: this.player.c - dc },
    to: { r: r, c: c }, boxMoves: boxMove ? [boxMove] : [], dir: this.dir }
  this.animating = true
}

```

- pages/game/game.js 实现目标点角色的呼吸脉动动画。

```

if (v === 3) {
  if (this.imgReady && this.imgs.pig) {
    const c = this.cell
    const cx = this.offsetX + j * c + c / 2
    const cy = this.offsetY + i * c + c / 2
    const pulse = 1 + Math.sin(now / 380 + i * 0.9 + j * 0.9) * 0.05
    ctx.save()
    ctx.translate(cx, cy)
    ctx.scale(pulse, pulse)
    ctx.drawImage(this.imgs.pig, -c / 2, -c / 2, c, c)
    ctx.restore()
  }
}

```

- pages/game/game.js 实现主角的镜像翻转与动态眼睛。向左移动时面向系数为 -1，整只角色平滑镜像，眼睛在缩放上下文中绘制并随方向转动。

```

const flipx = this.getFacing(now)
ctx.save()
ctx.translate(cx, cy - lift)
ctx.scale(flipx * (2 - stretch) * breathe, stretch * breathe)
ctx.drawImage(this.imgs.bird, -c / 2, -c / 2, c, c)
this.drawBirdEyes(c, dir, flipx)
ctx.restore()

```

- pages/game/game.js 实现目标点角色的语气词气泡与推箱尘土特效。

```

spawnPigBubble: function (r, c) {
  const list = ['呃啊!', '哎哟!', '轻点轻点!', '被压扁啦~', '救命呀!']
  const text = list[Math.floor(Math.random() * list.length)]
  this.bubbles.push({ r: r, c: c, text: text, t0: Date.now(), dur: 1400 })
  this.playSfx('bubble')
  if (this.bubbles.length > 3) this.bubbles.shift()
}

```

```

spawnPushFx: function (boxMove, dr, dc) {
  const c = this.cell
  const bx = this.offsetX + (boxMove.fc + 0.5) * c
  const by = this.offsetY + (boxMove.fr + 0.5) * c
  for (let k = 0; k < 11; k++) {
    this.fx.push({ type: 'dust', x: bx, y: by,
      vx: -dc * (20 + Math.random() * 60), vy: -dr * (20 + Math.random() * 60) - 20,
      life: 1, decay: 0.022 + Math.random() * 0.02,
      size: c * (0.07 + Math.random() * 0.09), color: 'rgba(160, 128, 92, ' })
  }
  this.fx.push({ type: 'ring', x: bx, y: by, life: 1, decay: 0.07,
    radius: c * 0.18, maxR: c * 0.62 })
}

```

- pages/game/game.js 初始化音效与背景音乐，使用微信音频接口循环播放背景音乐。

```

initAudio: function () {
  const self = this
  this.audio = {}
  const names = ['move', 'push', 'bump', 'click', 'bubble', 'win']
  names.forEach(function (name) {
    const ctx = wx.createInnerAudioContext()
    ctx.src = '/audio/' + name + '.wav'
    ctx.volume = 0.8
    self.audio[name] = ctx
  })
  this.bgm = wx.createInnerAudioContext()
  this.bgm.src = '/audio/bgml.wav'
  this.bgm.loop = true
  this.bgm.volume = 0.45
  this.audioReady = true
}

```


二、问题总结与体会

问题及解决

1. 自定义导航栏按钮被微信胶囊按钮遮挡

启用沉浸式自定义导航栏后，微信自带的胶囊按钮位于屏幕右上角，自定义按钮放置其上会被遮挡。解决方式为调用接口获取胶囊按钮的位置，动态计算导航栏右侧留白，将功能按钮推到胶囊按钮左侧。

2. 主角图片上叠加的动态眼睛位置不准

最初按正面脸的直觉把眼睛绘制在素材中央，但该素材为侧身绘制，两只眼睛均偏右，导致错位。解决方式为对素材图片逐像素分析，定位两只眼睛的真实坐标，并据此修正绘制位置；同时将眼睛绘制移入角色的缩放上下文，使跳跃拉伸时眼睛始终贴合角色。

3. 镜像翻转后瞳孔方向反向

为主角增加向左镜像翻转后，翻转使局部坐标左右互换，瞳孔的偏移方向随之反向。解决方式为记录面向系数，当面向系数为负时对瞳孔的水平偏移取反，保证屏幕上始终指向真实移动方向。

4. 背景音乐文件体积较大

合成的背景音乐为 wav 格式，体积约 600KB，加上图片素材后需控制在微信小程序 2MB 的包体积限制内。解决方式为精简背景音乐时长并控制各音效文件体积，最终代码包约 900KB，预览正常。

5. 部分平台限制音频自动播放

页面加载时直接播放背景音乐在部分平台会被拦截。解决方式为将背景音乐的启动置于用户手势之后，即点击剧情弹窗中的“开始游戏”按钮时才开始播放；页面切后台自动暂停，返回时恢复，离开页面时销毁音频上下文，避免资源泄漏。

6. 移动过程缺乏过渡动画

初始版本中角色移动为直接切换位置，画面生硬。解决方式为引入基于 `requestAnimationFrame` 的连续渲染循环，移动时平滑滑动并伴随跳跃拉伸，待机时目标角色呼吸、主角轻微浮动，推箱时的尘土、波纹、气泡与星形标记等特效统一由该循环驱动，画面明显生动。

收获与体会

对比之前信息展示类小程序，本次实验以教材推箱子基础骨架为起点，自主完成多项拓展功能。我除了适配新版 Canvas2D，处理设备像素比适配，使用 `requestAnimationFrame` 搭建渲染循环，还实现了一些拓展亮点：主角动态眼睛与左右镜像翻转、推箱粒子尘土特效、剧情弹窗、小猪随机气泡、音效震动、关卡解锁、撤销、通关彩带与数字滚动等等。

调试时碰到很多细节难题：侧身小鸟眼睛坐标错位、镜像翻转造成瞳孔方向出错、音频受平台限制只能手势触发、导航栏要规避胶囊按钮遮挡。我意识到游戏开发不只保证基础功能可用，画面反馈、真机适配都需要打磨，很多特效调试工作量大于教材原始逻辑。

项目采用数据驱动管理地图与剧情，本地存储记录游戏进度，拓展关卡十分便捷，同时校验关卡可行性防止死局。本次实践综合运用 Canvas、动画循环、存储、音频等能力，加深了我对小程序游戏开发的理解。